

Best Practices for Machine Readable Data and Code Sharing in Psychology

Daniël Lakens¹, Cristian Mesquida², and Lisa DeBruine³

¹Industrial Engineering and Innovation Sciences, Eindhoven University of Technology

²Industrial Engineering and Innovation Sciences, Eindhoven University of Technology

³Institute of Neuroscience and Psychology, University of Glasgow

Author Note

Daniël Lakens  <https://orcid.org/0000-0002-8393-5316>

Cristian Mesquida  <https://orcid.org/0000-0002-1542-8355>

Lisa DeBruine  <https://orcid.org/0000-0002-7523-5539>

A reproducible version of this manuscript available at on GitHub (https://github.com/scienceverse/machine_readable_FAIR_data_and_code). The authors have no conflict of interest to declare. This work was funded with the support of the Netherlands Organization for Scientific Research (NWO Vici grant VI.C.241.013). Author roles were classified using the Contributor Role Taxonomy (CRediT; <https://credit.niso.org/>) as follows: Daniël Lakens: conceptualization, writing, funding acquisition; Cristian Mesquida: conceptualization, writing; Lisa DeBruine: conceptualization, writing

Correspondence concerning this article should be addressed to Daniël Lakens, Industrial Engineering and Innovation Sciences, Eindhoven University of Technology, Den Dolech 2, 5600 MB, Eindhoven, Email: D.Lakens@tue.nl

Abstract

Distinguish necessary from optional metadata. How do codebooks deal with units? Variable types

Keywords: FAIR, Data sharing, Reproducibility, Open Science

Best Practices for Machine Readable Data and Code Sharing in Psychology

The [European Code of Conduct for Research Integrity](#) states: “Researchers, research institutions, and organisations ensure that access to data is as open as possible, as closed as necessary, and where appropriate in line with the FAIR Principles (Findable, Accessible, Interoperable and Reusable) for data management.” The [Dutch Code of Conduct for Research Integrity](#) states that: “Ensure that, in accordance with the FAIR principles, data is open and accessible to the extent possible and remains confidential to the extent necessary.” The FAIR principles state that research materials should be Findable, Accessible, Interoperable, and Reusable ([Wilkinson et al., 2016](#)). While there might be situations where a person would manually download and look at the data and code that has been shared, it is much more likely that others will access data and code through a machine. In most cases, data and code will have to be found through a search engine. That means that data is most re-usable when it is machine readable ([Lakens & DeBruine, 2021](#)). This guide to data and code sharing aims to set out best practices to make data and code findable, accessible, interoperable, and re-usable by machines.

Having invested considerable time and effort trying to re-use data and code files that researchers have shared when working on [Metacheck](#) we have developed an principled set of rules for data and code sharing that - if followed - will make it possible for machines to re-use your data effortlessly. This guide is based on the following principles. 1) If data and code is shared well, all aspects of the data and code can be retrieved with open source software, 2) data and code can be processed without error through rule-based algorithms, and 3) data and code is shared by adhering to existing standards if these have been developed. If data and code is not shared well, it's processing will rely on large language models (LLMs), and even with LLMs not all data might be accessible without error. This guide offers best practices in data and code sharing should enable every file to be found, listed, downloaded, read, classified, placed, and described in full without any uncertainty, without the need for guesses or the use of LLMs, or machine learning classifiers. This guide is written for researchers and data managers. We will discuss, repository organization, files naming, file formats, codebooks, code, readme and requirement files, data

layout, column naming, codebook generation, specifying scales and measures, and adhering to existing schemas for each.

1 Repository Choice

Before a files can be read, a tool has to find the data and code, list it's contents, and download it. There are many data repositories, each with strengths and weaknesses. Some data repositories used in psychology are Zenodo, GitHub, the Open Science Framework, ResearchBox, and PsyArchives. [ResearchBox](#) asks users to provide some meta-data when uploading files, which can facilitate - but is typically not sufficient - to classify files. ResearchBox backs up publicly available files as a .zip on Zenodo. Files on GitHub can be changed, and are therefore not a stable record, but it is straightforward to create a [release that is also archived on Zenodo](#). Zenodo is hosted by CERN and commissioned by the European Commission through the OpenAIRE project. The [Open Science Framework](#) is hosted by the Center for Open Science. [PsyArchives](#) is hosted by ZPID, and allows researchers to share their files under restrictions. While public files on PsychArchives can be downloaded automatically, files shared under more restrictive licenses are not accessible by a machine. The OSF, Zenodo, and GitHub allow users to access files through an public API. Files on PsychArchives can als be accessed through an API, even though it is not publicly described. ResearchBox does not have an API yet, which is an important limitation for machine accessibility. The R package Metacheck contains a `repo_check()` module that resolves links to OSF, GitHub, Zenodo, and PsychArchives, to their respective APIs, lists every file with its size and type, and flags components that are private, embargoed, or otherwise inaccessible to a machine, and its `code_check` and `data_check` modules can download files selectively (e.g., only all data and code files, but not all shared materials). The package can also scrape the ResearchBox website and download the zip archive of all shared files.

PsychArchives, ResearchBox, and the Open Science Framework make it possible to share an anonymous link to the datafiles for peer review. GitHub repositories can be made anonymous for blind peer review through [Anonymous GitHub](#). Zenodo allows users to create a new upload in Zenodo, keep it private, fill in the metadata while not disclosing any personal information, and

without publishing the record, click Share, Links, and create a secret link with the permission that people with the link “Can preview draft”. This allows reviewers to access the unpublished draft, while the files remain invisible to the public. When shared publicly, Zenodo, PsychArchives, and the OSF allow users to create a Digital Object Identifier, which provides a stable link to the files. GitHub is in itself not a stable repository, as files can be changed or deleted. The same is true for files on the Open Science Framework, unless the files are part of a Registration. However, the use of the Registration function (which is intended for preregistrations) to create a stable archive is somewhat convoluted. Furthermore, API limitations for downloads make the OSF a less attractive choice than Zenodo or PsychArchives. PsychArchives is limited to psychology and related disciplines. If you work in psychology, it is a solid choice for a repository, and its requirement to provide extensive metadata might be effortful, but facilitates FAIR re-use, as long as files are shared publicly. Zenodo is often the best general-purpose archival repository, whether it is used directly, or through ResearchBox or a GitHub release linked to Zenodo, but domain-specific repositories can be preferable when they store more detailed metadata. Never share data on your personal homepage, or in a cloud storage service such as dropbox or OneDrive, as such files can’t be approached through an API, and are not stable.

2 Keep repositories as small as possible

Software that automatically downloads the files you share will need to incorporate file size limits and repository size limits to preserve disk space. In general, repositories should be as small as possible. Online data repositories often offer generous file size limits. For example, Zenodo allows users to share 50GB per record, in at most 100 files. However, if scientific papers would routinely share files that take up 50GB then the re-use of files would require large data storage solutions. Although some data and code might be very large, in general it should be possible to reduce the size of the files that are shared, while enabling re-use and error checking. Data repositories might contain files that researchers share to facilitate *replication* (i.e., materials and stimuli used in the study), which are not necessary to *re-use* data and code, for example to *reproduce* results. As long as materials are clearly labeled in a repository they can be skipped

during the download. Therefore, the file size of data and code should be as small as possible, but materials can take up more space.

The following general recommendations can be followed to reduce the file size of data and code, and facilitate machine readability. First, although somewhat arbitrary, try to keep the size of individual files below 100MB, and the file size of the entire repository below 500MB. These thresholds provide a balance between reasonably high upper limits for repositories that need them, and a clear cut-off that can be managed (for example by compressing files). Metacheck's `download_repo_files()` applies these file and repository limits by default, but allows users to adjust the size limits. If .csv files are very large, the file size can be substantially reduced by compressing them into a .gzip format. As .gzip files can still be read in directly without needing to unzip the archive, this file format should be used for .csv files larger than approximately 10 MB to reduce size of the repository. For other large files it can be beneficial to compress them as a .zip archive.

Although it might be tempting to include links to GitHub repositories that contain software that was used or is related to the topic of the manuscript in the main text of an article, this is a bad practice as machines will not be able to distinguish links to your data and code on GitHub from a link to software. The machine will then have to guess whether the files on GitHub are data and code, or software files, which should be avoided. Instead, provide a citation to the repository, and such that the link to the GitHub repository appears only in the references. This prevents tools from making thousands of API calls to GitHub repositories with software in an attempt to classify all files in the repository.

Some files can be completely reproduced by other files in a repository. For example, code for computer simulations might create a large .csv file with results over hundreds of thousands of simulations. Labeling such files as 'output' will make it possible to download them only when needed. Long-term storage of output comes at cost, so the general recommendation is that outputs should only be stored when they substantially reduce computational burden or facilitate verification. To increase computational reproducibility, R projects should include an `renv.lock` file

and Python projects a requirements.txt or environment.yml file. For analyses that depend on system libraries or software beyond R or Python packages, containerized solutions such as Docker, Nix, or Code Ocean provide stronger guarantees of long-term reproducibility.

3 File and Folder Naming Conventions

Folders are used to group related files. For scientific papers, folders indicate projects, such as single studies, and categories of files (such as data, code, materials, or documents). The top level folder is associated with a paper, and it contains files that apply to all sub-folders in the repository. An example of a file that belongs to the top level folder is a readme - a text document that provides a human-readable description of the contents of the repository, and instructions on how to re-use the contents. For repositories that contain empirical research projects, each project in which data was collected gets its own sub-folder that follows the Psych-DS format, a data structure for research in the social science. This means that for multi-study manuscripts the repository contains a sub-folder for each study (e.g., 'pilot-1', 'study-1', 'study-2'). As unanimous agreement on naming conventions is difficult to achieve, which folders need to be present, and how they will be named will vary between research projects and fields.

At a minimum, every folder for a single study should contain a 'data' folder. If data is analyzed with individual code files, these are placed in a 'code' folder. If data is stored as a .csv file, it will require a codebook ([Arslan, 2019](#)). Based on the principle that folders are used to group related files, the best place for a codebook is alongside the data file in the data folder. The folder might also contain an 'output' folder that contains the stored results from the analysis scripts. Some analysis software (e.g., JASP and Jamovi) store data, codebooks, code, and output in a single file (i.e., a .jasp or .omv file). This removes the need for additional files and folders, as a single .jasp or .omv file in a data folder will allow a machine to retrieve all the information it needs. If all analyses that you want to perform are available in JASP or Jamovi, the easiest solution for machine readable FAIR data and code sharing is to import the data, enter the codebook information, perform the analyses, and share the .jasp or .omv file. This will most often be an easier and more reproducible solution than performing your analyses in R, where sharing

data and code equally FAIR requires substantially more work and expertise.

In addition to a data and code folder, repositories might contain a ‘documentation’ folder (for the preregistration document, the manuscript itself, etc.), and a ‘materials’ folder (for stimuli, survey files, etc.). Software will likely search for each of these folders under a range of aliases (e.g., the documentation folder might be called ‘reports’ or ‘docs’, the materials folder might be called ‘assets’ or ‘stimuli’, etc.). As long as the variation in file names is small, software can search for a list of folder names that contain a specific file type. Naming study components already starts when a study is preregistered. If the study number is too uncertain at the time of registration (e.g., the preregistration might end up being pilot study 2, not study 1) use a name that will return in the paper and folder structure (e.g., ‘survey on open science practices’ or ‘classification under load experiment’).

Files and folders should be named to be human readable, and allow classification by machines. As files will often need to be classified on multiple properties (e.g., a unique identifier, and the type of file) a machine should be able to separate these properties in a file name. A good convention is to use an underscore to distinguish different properties in a file name. For example, the file name ‘study-01_data.csv’ allows a machine to identify the type of file (i.e., a file that contains data), and the unique identifier (i.e., study-01). For file classification, the use of underscores and dashes will not matter, and even CamelCase file names (e.g., ‘codePowerSimulationStudy1’) should allow a machine to classify files. What is essential, however, is that each file name includes a word that allows a machine to classify it into a category. For scientific data, the main data types are data, code, codebook, output, documentation, and materials. The file name is an important source of information for the classification, and it should include one of these six words (or a sufficiently similar word, such as data dictionary, stimuli, etc. Metacheck’s file classifier implements this scheme directly, classifying every file in a repository using filename and extension rules, with a large-language-model fallback reserved only for files that remain ambiguous after rule-based classification fails (i.e., when files are not named well).

Some software might interpret names with and without a capital (e.g., Data.csv versus data.csv) as different files. Be consistent in the use of capitals. In general, it is recommended to only use lowercase letters, unless schemas and other data standards require capitalized filenames. Do not use spaces or special characters in filenames. Allow for lexicographic sorting of numbered files (e.g., if 'data-2' precedes 'data-11' name the files 'data-002' and 'data-011'). If you plan to limit the number of files in a repository to 99 (a requirement on Zenodo), two digits should be sufficient (but studies with a large number stimuli, or individual data files per participant, might require more digits for the files inside a .zip archive). Dates should be provided in YYYYMMDD format following the ISO 8601 basic calendar format (e.g., '20260724_data.csv').

Once files are found in a repository and listed (either based on the results of an API call, or after downloading all files), files will need to be classified into a type. File names should also include a descriptive keyword (e.g., 'preprint_documentation.pdf'). The use of subfolders can lead to long file and folder names. Certain file systems and software tools limit filenames at 255 characters. Furthermore, a folder needs to be downloaded to an existing folder, which adds to the total length of the filename. For example, if one wants to download a repository to C:\Users\dlakens\Downloads the filenames in the repository should remain below 228 characters. Try to keep the directory names (e.g., 'study-1-representative-sample/individual-participant-files/accuracy/') below 100 characters, and file names below 50 characters, so that there are around 100 characters left for the local folder name.

Names should match the information in the manuscript. If the manuscript refers to 'Study 1' the folder in the survey should be named 'Study 1', and not 'Survey 1' or 'Aggression Study'. This allows machines to link data and code in multi-study papers to the correct section of the manuscript.

4 Describe the Content and Organization of the Files in the Repository

A common recommendation for data and code sharing is to add a README document in every repository. A README document might be useful for people who manually browse

through the contents of a repository, but they are not very useful for machines. A README should be complemented by a `ro-crate-metadata.json` file, which is the structured counterpart to a traditional README, and include information that can't easily be retrieved from the files in the repository, while being of general interest to re-users. We suggest to add a `ro-crate-metadata.json` file at the root of the repository which follows existing schemas, specifically [RO-Crate 1.3](#) JSON-LD, using [schema.org](#), [DDI-Lifecycle 3.3](#) and [CRedit](#) terms. Metacheck's `convert_psychds()` function generates this file automatically: it writes an RO-Crate 1.3 JSON-LD graph populated with DDI-Lifecycle sample and data-handling fields, pre-filling what can be inferred from the repository and leaving an explicit 'unknown' placeholder for anything that requires researcher input.

The point of a README, or a corresponding RO-Crate meta-data file, is to provide information about the files in a repository that are difficult to retrieve from the files themselves. Information should be provided about which research project the data belongs to, and who created it. While researchers might think that re-users will find their files through the article they have written, machines are more likely to find repositories directly. Therefore, the meta-data needs to describe the project (and link to a DOI of the manuscript if one exists), including a description of the authors. Although good file naming should allow re-users to know what all files in a repository are, adding this information automatically to the JSON file serves as a validation that file names were unambiguous. As there is increasing interest in tracking whether manuscripts adhere to open science practices (for example to determine if funder requirements are met, or the Code for Conduct for Research Integrity has been followed), four boolean values track if a study has shared data, code, and materials, and whether the study is preregistered. These four practices are coded for each individual study, and indicate whether all data that can be shared has been made available, the code to reproduce all reported results has been shared, the materials to replicate a study have been made available, and whether the study has been preregistered in a study registry.

For each study, describe who the participants were, how they were reached, and how many participated. This information is often difficult to retrieve from a paper, but required to judge the

generalizability of the findings, and evaluate the strength of the evidence. Report the eligibility criteria, and inclusion of exclusion criteria. State the sampling method to describe how participants were recruited, and the sampling frame to describe the actual list you sampled from. Report the target sample size (which is for example based on a preregistered power analysis), the sample size you approached or randomised into a condition, the number of completed responses, and the resulting response rate. In a lab experiment where nobody drops out and the target sample size was reached these will be the same number, but if they diverge, they inform re-users about non-response. Because different experiments in one paper usually have different samples, this information is recorded once per study rather than once per paper.

Describe how a study's data came into existence. There is one section per study, and each section should be linked to its study folder. Describe the time window during which data was collected, the situation in which it was collected (for example, whether sessions were supervised, individual or group, or ran during an exam period), and the mode of collection (e.g., an online survey, in person interviews). Data handling does not stop at collection, and neither should the record of it. Following DDI's LifecycleEvent model each study lists its handling steps as a series of events, each stating what happened, when, and who was responsible. In a contributor field name anyone who handled the data but is not an author. This could be a research assistant who ran the sessions, a technician who exported the files, or a data manager who archived the files. These individuals should be named explicitly for two reasons. First, it is a way to make this labour visible. Second, it clearly logs who is responsible for each data processing step in the case of suspicions of research integrity violations. Every script that reads or writes data gets an event of its own that specifies the order in which files should be run. If data contains missing values, they should be coded in a consistent manner across datasets, and the coding scheme for missing values should be specified. This section should also list authors, using their ORCID ID, and the way they contributed to the paper following the CRediT taxonomy.

The suggestions so far Nearly are drawn from existing standards, such as the DDI-Lifecycle for the study and sample and data-handling descriptions, CRediT for contributor

roles, and schema.org for bibliographic metadata. We suggest four fields to log open science practices (`metacheck:hasSharedData`, `metacheck:hasSharedCode`, `metacheck:hasSharedMaterials`, `metacheck:isPreregistered`), as these do not yet have an equivalent in an existing vocabulary, but are increasingly tracked in order to recognize scholars who engage in open science practices.

```
{
  "@context": [
    "https://w3id.org/ro/crate/1.3/context",
    {
      "metacheck": "https://scienceverse.org/schema/metacheck/terms/",
      "ddi": "https://ddialliance.org/Specification/DDI-Lifecycle/3.3/",
      "populationOfConcern": "ddi:PopulationOfConcern",
      "samplingMethod": "ddi:SamplingProcedure",
      "samplingFrame": "ddi:SampleFrame",
      "targetSampleSize": "ddi:OverallTargetSampleSize",
      "obtainedDate": "ddi:DataCollectionDate",
      "modeOfCollection": "ddi:ModeOfCollection",
      "collectionSituation": "ddi:CollectionSituation",
      "eventType": "ddi:EventType",
      "dataCollectionMethodology": "ddi:DataCollectionMethodology",
      "purpose": "ddi:Purpose",
      "sampleSize": "ddi:SampleSize",
      "numberOfResponses": "ddi:NumberOfResponses",
      "specificResponseRate": "ddi:SpecificResponseRate",
      "isInclusive": "ddi:IsInclusive"
    }
  ],
  "@graph": [
    {
      "@id": "ro-crate-metadata.json",
      "@type": "CreativeWork",
      "conformsTo": { "@id": "https://w3id.org/ro/crate/1.3" },
      "about": { "@id": "./" }
    },
    {
      "@id": "./",
      "@type": "Dataset",
      "name": "Sleep Deprivation and Moral Judgment: Two Experiments",
      "version": "1.0.0",
      "dateModified": "2026-07-26",

```

```

"description": "Two pre-registered experiments testing whether acute sleep deprivation affects moral judgment",
"funder": "Dutch Research Council (NWO), grant VI.Vidi.000.000",
"author": [
  { "@id": "https://orcid.org/0000-0002-0247-239X" },
  { "@id": "#author-2" }
],
"identifier": "https://doi.org/10.1525/collabra.99999",
"keywords": ["sleep deprivation", "moral judgment", "dual process"],
"metacheck:hasSharedData": true,
"metacheck:hasSharedCode": true,
"metacheck:hasSharedMaterials": false,
"metacheck:isPreregistered": true,
"dateCreated": "2026-07-26",
"about": [{ "@id": "#missingvalues" }],
"hasPart": [
  {
    "@id": "study-ex1/",
    "@type": "Dataset",
    "name": "Study EX1",
    "about": [
      { "@id": "#study-design-ex1" },
      { "@id": "#lifecycle-ex1-collection" },
      { "@id": "#lifecycle-ex1-cleaning" },
      { "@id": "#lifecycle-ex1-analysis" }
    ]
  },
  {
    "@id": "study-ex2/",
    "@type": "Dataset",
    "name": "Study EX2",
    "about": [
      { "@id": "#study-design-ex2" },
      { "@id": "#lifecycle-ex2-collection" },
      { "@id": "#lifecycle-ex2-cleaning" },
      { "@id": "#lifecycle-ex2-analysis" }
    ]
  },
  {
    "@id": "documentation/collabra.99999_fulltext.txt",
    "@type": "CreativeWork",
    "name": "collabra.99999_fulltext.txt"
  },
  {
    "@id": "logs/collabra.99999.checks.json",

```

```

        "@type": "CreativeWork",
        "name": "collabra.99999.checks.json"
    }
]
},
{
    "@id": "#study-design-ex1",
    "@type": "ResearchProject",
    "name": "Study design (Study EX1)",
    "purpose": ["To test whether 24-hour sleep deprivation increases endorsement of ut
    "dataCollectionMethodology": ["Primary quantitative data collection; two-group bet
    "populationOfConcern": ["Adults aged 18-35 reporting habitual sleep of 7-9h, exclu
    "isInclusive": false,
    "samplingMethod": ["Convenience sampling from a university participant pool, rand
    "samplingFrame": ["SONA participant pool, Faculty of Social Sciences, 2025-2026 co
    "targetSampleSize": ["200"],
    "sampleSize": ["200"],
    "numberOfResponses": ["188"],
    "specificResponseRate": ["0.94"]
},
{
    "@id": "#lifecycle-ex1-collection",
    "@type": "CreateAction",
    "name": "Raw data obtained: Study EX1",
    "eventType": "raw data obtained",
    "startTime": "2025-03-04",
    "agent": { "@id": "#author-2" },
    "obtainedDate": ["2025-03-04/2025-05-22"],
    "collectionSituation": ["Supervised individual lab sessions, overnight, during ter
    "modeOfCollection": ["Face-to-face lab session; PVT via jsPsych on lab machines; q
    "contributor": [{ "@id": "#contributor-1" }]
},
{
    "@id": "#lifecycle-ex1-cleaning",
    "@type": "CreateAction",
    "name": "Data cleaning: Study EX1",
    "eventType": "data cleaning",
    "startTime": "2025-06-02",
    "agent": { "@id": "#author-2" }
},
{
    "@id": "#lifecycle-ex1-analysis",
    "@type": "CreateAction",
    "name": "Data analysis: Study EX1",

```

```

    "eventType": "data analysis",
    "startTime": "2025-06-10",
    "agent": { "@id": "https://orcid.org/0000-0002-0247-239X" }
  },
  {
    "@id": "#lifecycle-01-clean-r",
    "@type": "CreateAction",
    "name": "Script run: 01_clean.R",
    "eventType": "script run",
    "startTime": "unknown",
    "agent": "unknown",
    "position": 1,
    "instrument": { "@id": "study-ex1/analysis/01_clean.R" },
    "object": [{ "@id": "study-ex1/data/raw_qualtrics.csv" }],
    "result": [{ "@id": "study-ex1/data/cleaned_data.csv" }]
  },
  {
    "@id": "#lifecycle-02-analyse-r",
    "@type": "CreateAction",
    "name": "Script run: 02_analyse.R",
    "eventType": "script run",
    "startTime": "unknown",
    "agent": "unknown",
    "position": 2,
    "instrument": { "@id": "study-ex1/analysis/02_analyse.R" },
    "object": [{ "@id": "study-ex1/data/cleaned_data.csv" }],
    "result": [{ "@id": "study-ex1/analysis/models.rds" }]
  },
  {
    "@id": "#missingvalues",
    "@type": "DefinedTermSet",
    "name": "Missing-value scheme",
    "description": "Sentinel codes used across this paper's data to denote a missing r",
    "hasDefinedTerm": [
      { "@type": "DefinedTerm", "termCode": "-99", "name": "Prefer not to answer" },
      { "@type": "DefinedTerm", "termCode": "-98", "name": "Not applicable" }
    ]
  },
  {
    "@id": "#role-ex1-investigation",
    "@type": "Role",
    "name": "Investigation (Study EX1)",
    "roleName": { "@id": "https://credit.niso.org/contributor-roles/investigation/" },
    "about": { "@id": "study-ex1/" },

```

```

    "agent": { "@id": "#author-2" }
  },
  {
    "@id": "#role-ex2-validation",
    "@type": "Role",
    "name": "Validation (Study EX2)",
    "roleName": { "@id": "https://credit.niso.org/contributor-roles/validation/" },
    "about": { "@id": "study-ex2/" },
    "agent": { "@id": "https://orcid.org/0000-0002-0247-239X" }
  },
  {
    "@id": "https://orcid.org/0000-0002-0247-239X",
    "@type": "Person",
    "name": "Daniël Lakens",
    "affiliation": "Eindhoven University of Technology"
  },
  { "@id": "#author-2", "@type": "Person", "name": "Jane Doe" },
  { "@id": "#contributor-1", "@type": "Person", "name": "Sleepy", "affiliation": "Rese
]
}

```

Psych-DS Data Folders

In addition to the general meta-data discussed above, individual studies are represented as Psych-DS compliant sub-folders. The Psych-DS standard requires a `dataset_description.json`, which is a machine-readable codebook describing the contents of a specific dataset, including every variable, its measurement scale, its value range, the originating data file, and potentially additional information such as descriptive statistics. Unlike `ro-crate-metadata.json`, which describes the provenance, contributions, and general properties of the data and code, this file describes what the data mean, enabling software to automatically understand variables in datafiles. Machine-readable data descriptions supports findability, re-use and reproducible analyses. It allows users to search databases for studies that collected specific data. The Psych-DS JSON-LD document is based on Schema.org's Dataset model, extended with Psych-DS specific fields. These JSON files can be generated automatically from well-organized repositories, if they contain an accurate codebook, and use sensible file and folder names. Metacheck implements both directions of this standard: `psychds_check()` audits an existing repository against the Psych-DS 1.5

requirements and reports precisely which files are missing, misplaced, or need to be renamed, while `convert_psychds()` and the accompanying `psychds_validate()` generate a compliant `dataset_description.json` and independently re-validate it against the same rules used by the official validator.

5 File Formats

Data should be stored in open source formats, such as a .csv file, or software specific files, such as .svp, .jasp or .omv. If data is collected with commercial software, it is not always possible to make raw data (data in the form it is originally observed) available in open file formats. In such cases, the raw data files should be stored in a dedicated folder (“raw-data” inside the data folder. in a .zip archive is there are many files or if the files are large) for archival purposes, and exported as directly as possible to an open file format, such as csv. CSV files must be comma separated and follow the RFC 4180 specification, and utf-8 encoded. Any commas used within the data in a csv file itself must be enclosed by double-quotes (for example, an row with an open text entry would be stored as `pp1, 34, "To be, or not to be."`). Do not use plain .txt files for data storage, as machines do not automatically know how to process .txt files. Do not use .rdata files to store R output (as these can contain multiple objects, and a machine does not automatically know how to process is), but use .rds files instead. Best practice is to ensure that the file extension accurately reflects the underlying file format. For example, do not store a JSON formatted file in a .csv or .txt file, but save it as a .json file.

When using an archive, use .gzip for large csv files, and .zip for other files. A .zip archive stores its file index (the “central directory”) in the tail of the archive, so code can fetch only the last 128 KB of a .zip archive via an HTTP range request and read the name and size of every file inside the archive. This makes it possible to determine whether a .zip archive holds data worth downloading. For this reason the recommendation is to only use .zip files as an archive. Due to their proprietary format, .rar archives should not be used.

It is possible to store data in Excel spreadsheets (.xls or .xlsx) as this is an Open Office XML files that can be read in my open source software. However, certain practices should be

avoided. First, one should not store data in multiple tabs, as file names are used to classify datafiles, and when a spreadsheet contains multiple tabs, the name can't distinguish each dataset. Instead of using tabs, store multiple files. Second, data should be rectangular (a certain number of rows and columns) to be read in to other software. Spreadsheets with merged cells can't be read in as a dataframe, and spreadsheets with empty rows or columns can lead to problems. Leave truly empty cells empty, as software can replace empty excel cells with an empty value (e.g., as NA in R). Alternatively, use a coding scheme for missing values (e.g., 999 = 'no response', 888 = 'hardware failure'), and specify it in the `ro-crate-metadata.json` file. Third, do not rely on color coding, comments, or hyperlinks to communicate information. This information is lost when the data is read into other software, so always add such information in dedicated columns (e.g., a 'notes' column). CSV and Excel files must include headers in the first row, and only a single row can be used to label columns.

6 Codebooks and data dictionaries

A codebook or data dictionary describes datafiles. The two terms are used interchangeably, and a codebook and data dictionary largely contain the same information. Buchanan et al. (2021) note that a codebook customarily describes survey variables only, and can be made when data is collected, while a data dictionary is more commonly created in the planning stage, and additionally documents the file structure. Every datafile needs a codebook. Data files have a header of a single row that contains column names, and the codebook explains what these names stand for and what the values in the columns mean.

A complete codebook should contain six pieces of information. First, it contains the column name, exactly as it appears in the datafile (e.g., `panas-1`). Metacheck's `codebook_check()` automates this matching: it locates any codebook or labelled data file (including SPSS, Stata, SAS, JASP, jamovi, and Qualtrics formats), matches every data column to its codebook entry, and reports unmatched data columns, unused codebook entries, and cases where a codebook exists but does not align with the data file it is supposed to describe. Second, it contains a label that describes the column in prose (e.g., `positive and negative affect`

schedule item 1). If data did not originate from a survey question, describe what the values in the column are (E.g., reaction times, correct or false responses, reading times, etc). The goal of the label is to provide a human readable explanation of the variable, and add context. If the item is part of an existing scale, this is the place to mention the full name of the scale. A codebook at a minimum needs a column name and a label.

Third, for data with a limited number of response options, provide the value labels. For scales, these are how the response options were labels (e.g., 1 = very negative; 2 = somewhat negative; 3 = neutral; 4 = somewhat positive; 5 = very positive). There is no need to If only the endpoints are labeled, then only enter those two values (e.g., 1 = very negative; 5 = very positive). Fourth, for data which was generated based on survey questions, enter the verbatim question text (Interested). Fifth, if it is not possible to specify a single coding scheme for missing data across all data columns in the meta-data file of the repository (see above) then an individual column coding scheme for missing data can be included. Sixth, it can be useful to provide coding instructions in the codebook. One important coding instruction is reverse scored for items that should be reverse coded before combining into a scale mean. Other coding instructions can pertain to how the score will be used or has been used in the calculation of scales (e.g., mean of PANAS items 1-10) or that scores need to be transformed before analyzing.

The easiest way to create a codebook is to use data analysis software, and complete all codebook related fields in these tools. If your data are in SPSS (.sav), Stata (.dta), SAS (.sas7bdat), JASP (.jasp), jamovi (.omv), or Qualtrics (.qsf), fill in the variable and value labels inside that software and share the file. An important benefit of these file formats is that they combine the data and codebook in one file, which can be read without error. A limitation of these formats is that beyond .sav and .jaspfiles, the other formats do not allow users to specify missing values, and beyond Qualtrics, none of the formats allow specification of the exact item. Finally, none of these software package have a dedicated field for coding instructions. Therefore, if any of these additional fields are needed, users still need to create a dedicated codebook. If you are

working in R, it is possible to label dataframes, as R supports column label attributes (`attr(df$x, "label")`), but this option is not commonly used, and it is not recommended as some operations on dataframes can drop these attributes, and they are not as visible as a standalone codebook).

As a codebook has a tabular structure, and the column headers specify the variable information that is coded. An example of a table is provided below.

variable-name	label	values	missing- values	question	coding- instructions
pid	Participant identifier				
age	Age in years		-99	What is your age in years?	
gender	Self-reported gender	1 = Man; 2 = Woman; 3 = Non-binary	99	Which best describes your gender?	
bds_1	Breakup Distress item 1	1 (Strongly disagree) to 5 (Strongly agree)	99	I feel overwhelmed by the breakup.	
bds_3	Breakup Distress item 3	1 (Strongly disagree) to 5 (Strongly agree)	99	I am coping well.	reverse scored: 6 - bds_3

variable-name	label	values	missing- values	question	coding- instructions
bds_mean	Breakup Distress mean score				mean of bds_1 to bds_10 after reversing bds_3

You do not need to use these exact column headers for a codebook, as most automated tools will scan for a range of plausible column names (e.g., instead of values you value labels, response options, or levels, among others). Values need at least two code = label pairs. For example, 1 = intervention; 2 = control condition. You can also write intervention = 1; control condition = 2. Where only one side is numeric, that side should be the code. If both sides are text I = intervention; C = control condition then which side is the code can be resolved by checking which values are actually used in the datafile. Although the use of = as separator between the code and the label, and ; as separator between different values is a good practice, other consistently used separators are possible (including simply a space, e.g., 1 intervention; 2 control condition. For Likert and semantic-differential items, every response option can be labeled, but it is also possible to only label the scale ends (e.g., 0 = Not at all; 100 = Very important, or 1 = very negative; 4 = neutral; 7 = very positive).

Although tabular codebooks can be typed out in a .csv file, it might be most convenient to use a spreadsheet to create the codebook. When creating the codebook, a row of column names can be copied from the data file, and transposed (i.e., flipped to from one row to one column) and pasted in the spreadsheet. This should guarantee that the column names in the datafile and codebook match perfectly. Make sure all values are stored as text - Excel might transform 1-5 into the date 5-Jan. The codebook can be stored as a .ods, .xlsx, or .csv file. The first two formats

are easier to read by a human, while csv files are more interoperable. Do not store codebooks as word documents or pdf files, as the information in these documents can't be parsed reliably by a machine. Name the file codebook or data-dictionary. If there is only one data file in the same folder, there is no further need to use the filename to match the codebook to a datafile, as the folder groups related files. But if a folder has multiple datafiles, use the filename to match each codebook to its corresponding datafile (e.g., `pilot-1a_data.csv` and `pilot-1a_codebook.csv`).

7 Tabular data files

Data files should be rectangular table, with one header row, one variable per column, and one observation per row. Analysis software such as SPSS, JASP and Jamovi will force these data formats on users, but when datafiles are created in spreadsheets it is very tempting to perform actions that deviate from this standard. It is common to see a header spanning multiple rows, empty columns or rows, or written notes at the bottom of a spreadsheet. Metacheck's `data_validate()` module checks column names for characters that break machine parsing, flags collisions that would arise once names are sanitized, screens for leading and trailing whitespace and inconsistent encoding, and separately screens open-text columns for personally identifying information.

The header contains column names. Column names need to match the values in the codebook exactly. Best practices for column names are more strict than best practices for file names, as only letters, digits, and underscores should be used (e.g., `participant_id`, `mean_reaction_time`). Column names are in analysis code. In R, referring to a column can be done with `df$participant_id`. If column names include a space or a dash the column name needs quoting (`df$`participant-id`` or `df$`participant id``). Special characters often need to be escaped to print, or will be automatically renamed when importing the data file in other software (which could lead to identical column names if column names only differ in a special character). This is also true if characters with diacritics are used. Keep column names shorter than 32 characters, as this is the limit for some analysis software packages. All column names should be unique.

Use column names that can be understood (e.g., do not use SE1 but `self_esteem_scale_1`, and never use VAR1). Where possible, use column names that match the words used to refer to these data columns in the manuscript. If columns belong together use the same prefix (e.g., `self_esteem_scale`) and increasing numbers (e.g., `_1`, `_2`, `_3`). If columns correspond to a scale with subscales, the labeling convention discussed so far can be repeated for the subscale (e.g., `panas_1_positive_1` for the positive affect subscale of the positive and negative affect schedule). Scale items should occupy adjacent positions in a dataframe. If multiple columns related to the same response, such as the response value and the response time for a question, clearly label the properties that each column contains (e.g., `mood_1_response`, `mood_2_response`, `mood_1_reactiontime`, `mood_2_reactiontime` instead of `mood_1_response`, `mood_1_reactiontime`, `mood_2_response`, `mood_2_response`).

There should be no columns or rows with exclusively empty values. Column names and values in columns should not contain leading or trailing whitespace (e.g., not `yes` or `yes`, but `yes`). If columns are almost completely numeric, do not add a single text value (e.g., ‘no response’) as it prevents the entire column from being read as numeric values. Carefully check if the data contains personal information (e.g., email addresses, IP addresses) and screen open text for identifying information.

8 Machine-readable measures and tasks

Codebooks describe variables in datafiles. Subsets of these variables hold special importance to scientific communities, such as responses to validated scales and widely used paradigms. Efforts have been made to create standards to share scales and tasks in more standardized ways. These data formats complement codebooks and allow re-users to connect information across data repositories. Two open machine-readable formats exist to share scales and behavioral task metadata: Open Scale Definition (OSD) for questionnaires and rating instruments, and the Behavior Data Model for trial-level behavioural data ([Defossez et al., 2020](#)). Metacheck operationalizes this pipeline end to end, `codebook_check()` identifies which validated scales and tasks are present in a dataset from column-name patterns, and `convert_psychds()` exports the

corresponding item and scoring metadata as Open Scale Definition files, cross-referenced from the automatically generated Psych-DS dataset description.

Open Scale Definition is a JSON format for encoding a psychological or behavioural measurement instrument — its items, response options, scoring rules, and citation — as a single portable, runner-agnostic file. It is part of the Psychology Experiment Building Language created for building and running psychological experiments ([Mueller & Piper, 2014](#)). An .osd file has two top-level parts: a definition (the structure, language-neutral) and translations (the actual text, keyed by language code, so the same structure can carry versions of a scale in different languages). The OSD specification requires `scale_info`, `likert_options`, `dimensions`, `items` and `scoring` (although an entry can be set to NULL). The scale info contains a name, code, and abbreviation, a citation to the original publication of the scale, license information, and other descriptive information. The JSON file also contains information about the likert-scale response options, the dimensions of the scale (such as sub-scales, and which items belong to which subscale), how the scale is scored, and the specific text for each item, in different translations if available. This information is too detailed to be part of a codebook (as not all columns in a datafile require the information that should be provided for scales), and a benefit of using a standardized scale available on the OpenScales project is that the meta-data file can simply be downloaded. FAIR scale meta-data also facilitates standardization in how scales are used and scored across studies. A minimal, worked example — one reverse-keyed 5-item scale with a single subscale:

```
{
  "osd_version": "1.1.0",
  "definition": {
    "scale_info": {
      "name": "Breakup Distress Scale",
      "code": "BDS",
      "citation": "Author, A. (2024).",
      "license": "CC BY 4.0",
      "version": "1.0",
      "domain": "Clinical"
    },

```

```

    "items": [
      {"id": "bds_1", "text_key": "bds_1", "type": "likert", "dimension": "distress"},
      {"id": "bds_2", "text_key": "bds_2", "type": "likert", "dimension": "distress"},
      {"id": "bds_3", "text_key": "bds_3", "type": "likert", "dimension": "distress", "r"},
      {"id": "bds_4", "text_key": "bds_4", "type": "likert", "dimension": "distress"},
      {"id": "bds_5", "text_key": "bds_5", "type": "likert", "dimension": "distress"}
    ],
    "likert_options": [
      {"text_key": "opt_1", "value": 1},
      {"text_key": "opt_2", "value": 2},
      {"text_key": "opt_3", "value": 3},
      {"text_key": "opt_4", "value": 4},
      {"text_key": "opt_5", "value": 5}
    ],
    "dimensions": [
      {"id": "distress", "name": "Breakup Distress", "description": "Overall distress ab"},
    ],
    "scoring": {
      "distress": {
        "method": "mean_coded",
        "items": ["bds_1", "bds_2", "bds_3", "bds_4", "bds_5"],
        "description": "Mean of all 5 items after reverse-scoring bds_3."
      }
    },
    "translations": {
      "en": {
        "bds_1": "I feel overwhelmed by the breakup.",
        "bds_2": "I think about the breakup constantly.",
        "bds_3": "I am coping well.",
        "bds_4": "The breakup has disrupted my daily life.",
        "bds_5": "I feel a sense of loss.",
        "opt_1": "Strongly disagree", "opt_2": "Disagree", "opt_3": "Neutral",
        "opt_4": "Agree", "opt_5": "Strongly agree"
      }
    }
  }
}

```

Where the Open Scale Definition describes a measurement instrument, the Behavior Data Model describes the data a participant produces while completing the measures, on the level of single trials. The schema's central table describes the response, for one row per trial or per response. Thirteen fields are required, namely response id, the study name, the participant

identifier, the session id (in case participants are tested more than once), the instrument id, whether the tasks belongs to a multi-task design, the block id, the block index, the block type, transformations that create a trial (often not needed in paradigms where each row is a single trial), the trial index, the date and time a trial started, and the stimulus id shown at a specific trial, and the stimulus type. Optional fields carry the substantive measurements a script actually analyses, such as the response time, accuracy, or score. A companion instrument table gives the abstract descriptor for the task or questionnaire that the trials belong to, including a link field that can point at the paired .osd file. A minimal example consisting of three trials of a Stroop task would look like:

```
{
  "Instrument": [
    {"instrument_id": "stroop", "timeline_id": "stroop", "block_id": "stroop",
     "name": "Color-Word Stroop Task", "link": "../scales/STROOP.osd"}
  ],
  "Response": [
    {"response_id": "1", "study_name": "study1", "agent_id": "p001", "session_id": 1,
     "instrument_id": "stroop", "multitask_type": "single_task", "block_index": 1,
     "block_type": "questionnaire", "transformation_name": "identity",
     "trial_index": "1", "trial_start_datetime": "2026-03-01T09:00:00Z",
     "stimulus_id": 1, "stimulus_type": "stimulus",
     "stimulus_description": "RED (in blue ink)", "response_description": "blue",
     "response_time": 612.4, "correct": true},
    {"response_id": "2", "study_name": "study1", "agent_id": "p001", "session_id": 1,
     "instrument_id": "stroop", "multitask_type": "single_task", "block_index": 1,
     "block_type": "questionnaire", "transformation_name": "identity",
     "trial_index": "2", "trial_start_datetime": "2026-03-01T09:00:01Z",
     "stimulus_id": 2, "stimulus_type": "stimulus",
     "stimulus_description": "GREEN (in green ink)", "response_description": "green",
     "response_time": 487.1, "correct": true},
    {"response_id": "3", "study_name": "study1", "agent_id": "p001", "session_id": 1,
     "instrument_id": "stroop", "multitask_type": "single_task", "block_index": 1,
     "block_type": "questionnaire", "transformation_name": "identity",
     "trial_index": "3", "trial_start_datetime": "2026-03-01T09:00:02Z",
     "stimulus_id": 3, "stimulus_type": "stimulus",
     "stimulus_description": "YELLOW (in red ink)", "response_description": "red",
     "response_time": 701.8, "correct": false}
  ]
}
```

```
}
```

Discussion

Data and code should be shared where possible to facilitate check and re-use. Repositories containing FAIR data and code should allow machines to find and access all content. This requires data and code to be made available in online repositories, that can feasibly be downloaded, where files and folders are clearly named to allow automatic categorization of files, the contents of the repository are described, files are shared in open formats, a codebook is included, datafiles are well-structured, and where relevant, meta-data about measures and tasks is provided. Schema's such as Psych-DS, RO-Crate JSON-LD, DDI-Lifecycle, CRediT, Open Scales Definition, and Behavense Data Model facilitate FAIR data and code.

Adhering to best practices for FAIR data and code is a complex and effortful task. There are over 100 recommendations for best practices in this document, and it is unreasonable to expect that researchers or data managers will remember and implement them all. A solution to overcome limitations in memory and training is to automate the generation of FAIR data and code as much as possible. Software solutions could be developed that scan a repository and assist researchers in iteratively improving how FAIR their data and code is. Metacheck (<https://github.com/scienceverse/metacheck>) is one implementation of this approach: it scans a repository, classifies its files, checks them against Psych-DS, generates the corresponding RO-Crate metadata, and reports — rather than silently assumes — which of the practices described in this guide are met, so that researchers only need to supply the information that cannot be inferred automatically. Input from researchers is essential to provide all required meta-data, but it can be explicitly requested only where needed. If data and code can be processed by a machine it becomes possible to validate the extent to which data and code is FAIR, and guide researchers towards improving the findability, accessibility, interoperability, and re-usability of their findings.

The potential of machine readable data and code is immense. By scanning data repositories all found datafiles can be processed automatically, and meta-data can be stored in a central database. For example, it would be easy to search the database for datafiles where

researchers have administered specific scales or tasks, while filtering for demographic information about the sample. This would enable the re-use of research data on a massive scale. Machine readable code and data would also facilitate automated reproducibility checks. This can increase the quality of scientific manuscripts, without increasing the burden on peer reviewers. Machine readable data and code will likely play an important role in the ability of researchers to adhere to the principles in the code of conduct for research integrity to share FAIR data and code when possible.

References

- Arslan, R. C. (2019). How to Automatically Document Data With the codebook Package to Facilitate Data Reuse. *Advances in Methods and Practices in Psychological Science*, 2(2), 169–187. <https://doi.org/10.1177/2515245919838783>
- Buchanan, E. M., Crain, S. E., Cunningham, A. L., Johnson, H. R., Stash, H., Papadatou-Pastou, M., Isager, P. M., Carlsson, R., & Aczel, B. (2021). Getting Started Creating Data Dictionaries: How to Create a Shareable Data Set. *Advances in Methods and Practices in Psychological Science*, 4(1), 2515245920928007. <https://doi.org/10.1177/2515245920928007>
- Defossez, A., Ansarinia, M., Clocher, B., Schmück, E., Schrater, P., & Cardoso-Leite, P. (2020). *The structure of behavioral data* (No. arXiv:2012.12583). arXiv. <https://doi.org/10.48550/arXiv.2012.12583>
- Lakens, D., & DeBruine, L. M. (2021). Improving Transparency, Falsifiability, and Rigor by Making Hypothesis Tests Machine-Readable. *Advances in Methods and Practices in Psychological Science*, 4(2), 2515245920970949. <https://doi.org/10.1177/2515245920970949>
- Mueller, S. T., & Piper, B. J. (2014). The Psychology Experiment Building Language (PEBL) and PEBL Test Battery. *Journal of Neuroscience Methods*, 222, 250–259. <https://doi.org/10.1016/j.jneumeth.2013.10.024>
- Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Santos, L. B. da S., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., ... Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship [Comments and {{Opinion}}]. In *Scientific Data*. <https://www.nature.com/articles/sdata201618>. <https://doi.org/10.1038/sdata.2016.18>